The 4-Step System Design Approach:
From Problem Definition to Final Solution

The System Design Blueprint

Mastering System Design: From Basics to Cracking Interviews

1

Understanding Problem & Scope

Define requirements, constraints and success criteria.



2

Estimating Scale & Bottlenecks

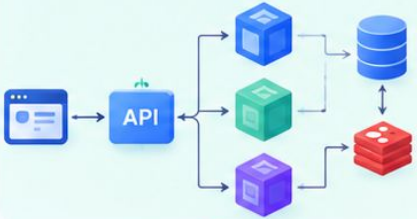
Estimate users, traffic, storage and identify bottlenecks early.



3

High-Level Design
Services, APIs & Communication

Design services, define APIs and plan communication between components.



4

Strategic Technology & Infrastructure Decisions

Choose the right technologies, infrastructure and deployment strategy.





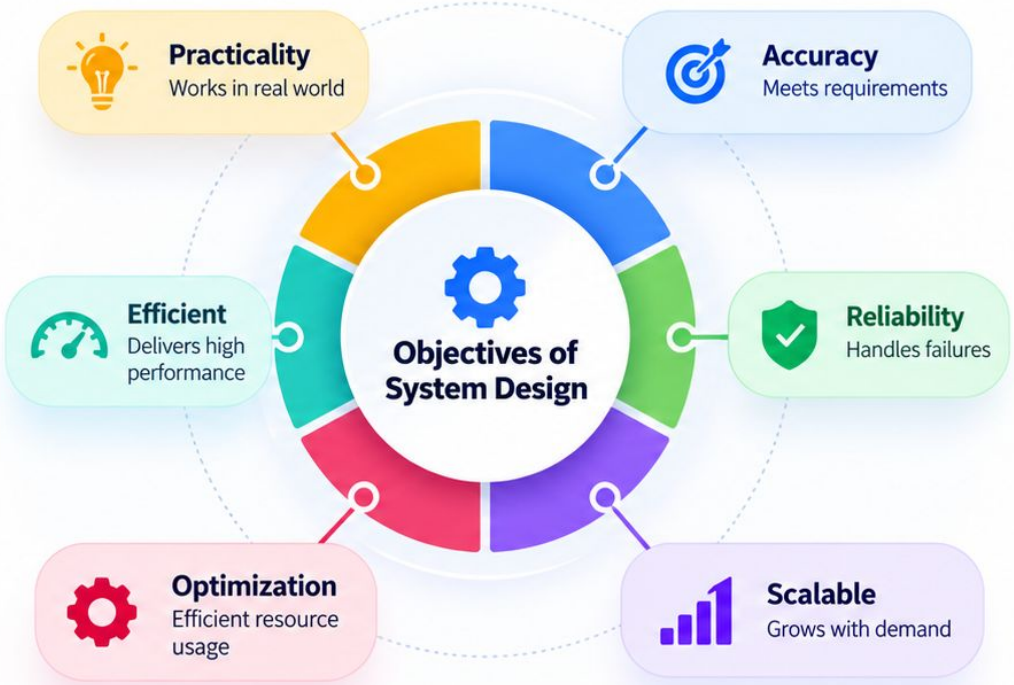
Instructor: Rahul Rajat Singh

What is System Design?

Building **scalable**, **maintainable** systems

Balancing trade-offs between **performance, cost,** and **complexity**

The importance of **clear architecture** and **thoughtful planning**



The 4-Step System Design Process

1 Understanding the Problem & Defining the Scope



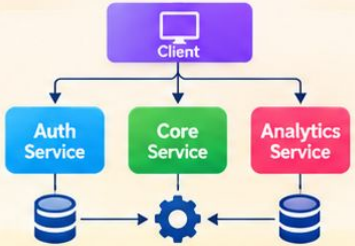
- **Functional Requirements:** Define the core features (e.g., URL shortening, redirection).
- **Non-Functional Requirements:** Consider performance, scalability, reliability, security.
- **Constraints:** Account for time, budget, regulatory, and technical limitations.

2 Estimating Scale & Identifying Bottlenecks



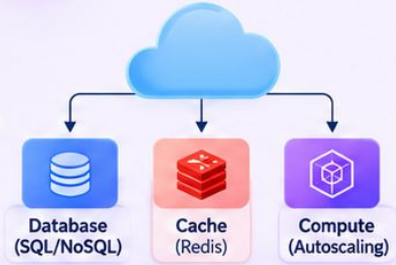
- **Estimating Traffic:** Analyze peak load, traffic patterns, and user growth.
- **Identifying Bottlenecks:** Pinpoint critical components (e.g., database, CPU, network) that may face issues.
- **Capacity Planning:** Estimate storage, bandwidth, and compute requirements to handle growth.

3 High-Level Design: Services, APIs & Communication



- **Core Services:** Break down the system into essential services (e.g., URL shortener, analytics).
- **API Design:** Define public-facing endpoints (e.g., POST /shorten, GET /redirect).
- **Communication Patterns:** Choose synchronous vs. asynchronous (e.g., REST APIs, WebSockets).
- **Service Interaction:** Define how services communicate (e.g., via APIs, message queues).

4 Making Tech & Infra Decisions Strategically



- **Tech Stack Decisions:** Choose between SQL/NoSQL, data stores, and caching solutions (e.g., Redis).
- **Scalability & Availability:** Implement load balancing, autoscaling, and replication.
- **Performance Considerations:** Focus on latency, throughput, and caching hot data.
- **Trade-offs:** Weigh cost vs. performance, simplicity vs. complexity (e.g., Redis for caching vs. traditional databases).

1 2 3 4

Instructor: Rahul Rajat Singh

Conclusion – The Blueprint for Success

1. Always start with a **clear understanding** of requirements and constraints.
2. Use **scale and bottleneck analysis** to guide architectural decisions.
3. Create a **high-level design** that balances performance, cost, and complexity.
4. Make informed **infrastructure and tech choices** to ensure scalability and availability.



Step 1
Requirements & Constraints
Understand the problem, define scope, and identify constraints.

Step 2
Scale & Bottlenecks
Estimate scale, identify bottlenecks, and analyze trade-offs.

Step 3
Services, APIs & Communication
Design high-level architecture, define services, APIs, and communication patterns.

Step 4
Technology & Infrastructure
Choose the right technology stack and infra, considering cost, scalability, and operability.

Scalable, Reliable & Cost Effective System